C A H

# Service Chat

**Statut : Implementation-ready**

# Historique des Révisions

| Ver. | Date | Description | Auteur |
|---|---|---|---|
| 1.0 | 30 mars 2026 | Création initiale du document | Équipe Ingénierie |
| 1.1 | 30 mars 2026 | Post-audit : alignement `platform_id` UUID Auth. Préfixe `/api/v1`. Convention identité. PK corrigées. Table `platform_capabilities`. `TIMESTAMPTZ`. Contrats inter-services. Matrice autorisation. Frontière Chat/Search. Garde-fous scalabilité. PlantUML annexe. | Équipe Architecture |
| 1.2 | 9 avril 2026 | Version finale : statut *Implementation-ready*. Matrice autorisation en forme canonique. Gouvernance capabilities : rôle admin précisé via Users. Mapping identité Chat→Notification. Règles opérateur transfert verrouillées. | Équipe Architecture |

# Table des matières

**Historique des Révisions** **1**

**1 Introduction** **4**
1.1 Objet du Document . . . 4
1.2 Destinataires . . . 4
1.3 Périmètre du Service . . . 4
1.4 Conventions écosystème AG Technologies . . . 4
1.5 Frontière Chat / Recherche . . . 4

**2 Contexte & Problématique** **5**
2.1 Pourquoi Ce Service Existe . . . 5
2.2 Problèmes Résolus . . . 5
2.3 Positionnement Architecture Globale . . . 5

**3 Vision & Objectifs** **5**
3.1 Concept du Service . . . 5
3.2 Objectifs MVP & Long Terme . . . 6
3.3 KPIs . . . 6

**4 Parties Prenantes** **6**
4.1 Équipe Technique . . . 6
4.2 Services Consommateurs . . . 6

**5 Besoins Fonctionnels** **6**
5.1 Module Conversations . . . 6
5.1.1 Types de Conversations . . . 7
5.1.2 Gestion des Conversations . . . 7
5.1.3 Gestion des Participants . . . 7
5.2 Module Messagerie . . . 7
5.2.1 Messages . . . 7
5.2.2 Réactions Emoji . . . 7
5.2.3 Accusés de Lecture . . . 7
5.3 Module Temps Réel (WebSocket) . . . 7
5.3.1 Typing . . . 7
5.3.2 Présence . . . 7
5.3.3 Reconnexion . . . 7
5.4 Module Transfert Bot → Humain . . . 7
5.5 Module Capacités par Plateforme . . . 8
5.5.1 Capacités optionnelles . . . 8
5.5.2 Capacités structurelles (toujours actives) . . . 8
5.5.3 Comportement désactivé . . . 8
5.5.4 Gouvernance des capacités (v1.2) . . . 8

**6 Besoins Non-Fonctionnels** **9**
6.1 Performance . . . 9
6.2 Sécurité . . . 9
6.3 Scalabilité . . . 9

6.4 Garde-fous . . . 9
6.5 Disponibilité . . . 9

**7 Architecture Technique** **9**
7.1 Stack Technique . . . 10
7.2 Intégrations Externes . . . 10

**8 Modélisation** **10**
8.1 Table `conversations` . . . 11
8.2 Table `conversation_participants` . . . 11
8.3 Table `messages` . . . 11
8.4 Table `message_reactions` . . . 12
8.5 Table `read_cursors` . . . 12
8.6 Table `transfers` . . . 13
8.7 Table `platform_capabilities` . . . 13

**9 Contrats d'API** **13**
9.1 Matrice d'Autorisation (v1.2) . . . 13
9.2 Endpoints REST . . . 14
9.3 Événements WebSocket . . . 15
9.3.1 Serveur → Client . . . 15
9.3.2 Client → Serveur . . . 15

**10 Contrats Inter-Services** **15**
10.1 Chat → Auth (JWT) . . . 15
10.2 Chat → Auth (S2S) . . . 15
10.3 Chat → Users . . . 15
10.4 Chat → Notification . . . 16
10.4.1 Templates Notification pré-requis . . . 16
10.5 Chatbot → Chat . . . 16
10.6 Chat → Média (futur) . . . 16

**11 Infrastructure** **16**
11.1 Composants . . . 16
11.2 Variables d'Environnement . . . 17
11.3 Docker Compose . . . 17

**12 Recommandations & Évolutions** **17**
12.1 Scaling WebSocket . . . 17
12.2 Pièces Jointes . . . 17
12.3 E2E . . . 18
12.4 Archivage . . . 18
12.5 Suppression . . . 18

**A Annexe A — Diagramme de Classes (PlantUML)** **19**

**B Annexe B — MLD (PlantUML)** **20**

**Annexe C — Glossaire** **22**

# 1 Introduction

## 1.1 Objet du Document

Ce document constitue le **Cahier des Charges Technique et Fonctionnel** du **Service Chat** d'AG Technologies, version 1.2 (*Implementation-ready*). Il définit les objectifs, fonctionnalités, architecture technique, contrats d'API et contrats inter-services.

## 1.2 Destinataires

| Destinataire | Rôle |
|---|---|
| Équipe Backend | Développement et maintenance |
| Équipe Frontend | Intégration WebSocket, UI chat |
| Équipes consommatrices | AGT-Market, AGT-Bot, SALMA, Service Chatbot |
| Lead Technique | Validation des choix techniques |
| Chef de Projet | Suivi et validation fonctionnelle |

## 1.3 Périmètre du Service

**Périmètre inclus :** conversations directes, canaux et transferts bot→humain ; messagerie temps réel via WebSocket ; réactions emoji ; accusés de lecture ; indicateur de saisie ; présence en ligne ; recherche full-text intra-conversation ; historique paginé ; configuration de capacités par plateforme ; API REST et événements WebSocket complets.

**Hors périmètre :** authentification (Service Auth v2.1), stockage fichiers (Service Média v1.4), recherche transverse données métier (Service Recherche v1.2), logique métier plateformes, facturation, modération de contenu (futur).

## 1.4 Conventions écosystème AG Technologies

Conventions **non négociables** (validées Auth v2.1, Users v1.0, Notification v1.2, Média v1.4, Recherche v1.2) :

- `platform_id` = UUID du registre Auth (`platforms.id`)
- Identité utilisateur = `users_auth.id` (= `sub` du JWT)
- JWT validé via `GET /auth/verify-token` ; token S2S via `POST /auth/s2s/introspect`
- Préfixe API : `/api/v1`
- Rôles et permissions gérés par Users, pas par Auth
- RGPD orchestré par Users
- Stack : Django/DRF pour Auth, Users, Notif, Recherche ; Node.js/NestJS pour Média ; **Node.js/Express/Socket.io pour Chat** (justification : section 7.1)
- Timestamps : `TIMESTAMPTZ`

## 1.5 Frontière Chat / Recherche

**Règle non négociable :** La recherche full-text intra-conversation reste locale au

> Service Chat (index GIN PostgreSQL). Le Service Recherche (v1.2) n'indexe jamais de messages de chat.

# 2 Contexte & Problématique

## 2.1 Pourquoi Ce Service Existe

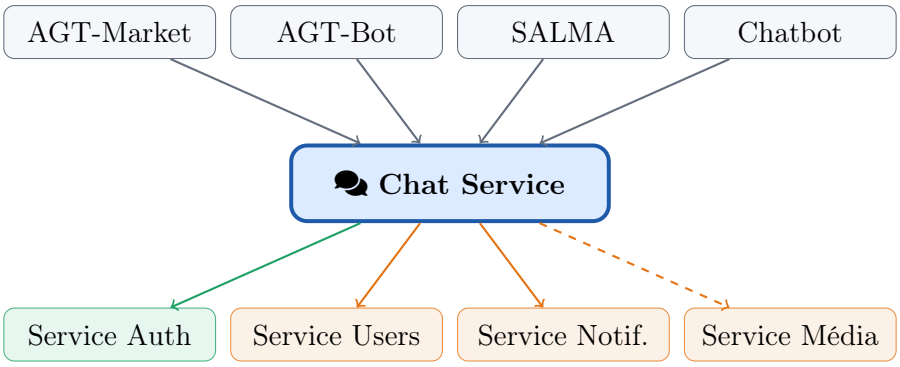
Dans un écosystème multi-plateformes, la communication entre utilisateurs est un besoin transverse critique. Sans service centralisé : duplication temps réel, incohérence fonctionnalités, impossibilité transfert bot→humain, problèmes scalabilité WebSocket, absence historique unifié.

## 2.2 Problèmes Résolus

| Problème | Solution |
|---|---|
| WebSocket par plateforme | Gateway centralisé avec Redis Adapter |
| Pas de transfert bot→humain | Type conversation dédié + file d'attente |
| Scalabilité WebSocket | Redis Pub/Sub + Socket.io Adapter multi-instances |
| Messages perdus hors ligne | Persistance PostgreSQL + livraison à la reconnexion |
| Concepts métier codés en dur | Service générique + capacités configurables par plateforme |

## 2.3 Positionnement Architecture Globale

Le Service Chat dépend du Service Auth (v2.1), Users (v1.0), Notification (v1.2) et, à terme, Média (v1.4).




# 3 Vision & Objectifs

## 3.1 Concept du Service

Le Service Chat est le système de messagerie temps réel unifié pour toutes les plateformes. Il est générique : trois types abstraits (directe, canal, transfert), métadonnées libres, capacités activables par plateforme.

### 3.2 Objectifs MVP & Long Terme

| Phase | Objectifs |
|---|---|
| **MVP** | Conversations directes/canaux ; reply, edit, soft delete ; accusés lecture ; typing ; présence ; réactions ; transfert bot→humain ; notifications hors ligne ; capabilities par plateforme ; API REST + WS |
| **Long terme** | Pièces jointes Média ; E2E ; archivage ; modération ; messages vocaux |

### 3.3 KPIs

| KPI | MVP | Long Terme |
|---|---|---|
| Latence WS | $< 200$ms | $< 100$ms |
| Connexions WS | 1 000+ | 10 000+ |
| Disponibilité | $\geq 99\%$ | $\geq 99{,}9\%$ |
| Perte messages | 0% | 0% |
| Prise en charge transfert | $< 5$ min | $< 2$ min |
| Taux erreur API | $< 1\%$ | $< 0{,}5\%$ |

## 4 Parties Prenantes

### 4.1 Équipe Technique

| Rôle | Responsabilités |
|---|---|
| Ingénieur Backend | API REST, conversations, messaging |
| Ingénieur Temps Réel | Gateway WS, Socket.io, Redis Pub/Sub |
| Lead Technique | Architecture, scaling, décisions techniques |
| Ingénieur DevOps | Redis, PostgreSQL, Docker, LB |
| Ingénieur Frontend | Intégration Socket.io, UI |

### 4.2 Services Consommateurs

| Service | Usage |
|---|---|
| AGT-Market | Chat acheteur↔vendeur, canaux boutique |
| AGT-Bot | Conversations user↔opérateur, transfert bot→humain |
| SALMA | Chat étudiant↔formateur, canaux cours |
| Service Chatbot | Création transferts (auth S2S) |
| Service Notification | Alertes push/email hors ligne |

## 5 Besoins Fonctionnels

### 5.1 Module Conversations

### 5.1.1 Types de Conversations

| Type | Caractéristiques |
|---|---|
| `direct` | 2 participants ; anti-doublon (mêmes participants + plateforme = même conversation) |
| `canal` | Illimité ; public/privé ; nom, description, propriétaire |
| `transfert` | Via Chatbot (S2S) ; statuts : `en_attente`, `pris_en_charge`, `cloture` |

### 5.1.2 Gestion des Conversations

- Création (type, participants, plateforme, métadonnées)
- Listage paginé (filtré type, plateforme)
- Détail avec dernier message + non-lus
- Modification canal (nom, description, visibilité)
- Soft delete ; tri par dernière activité

### 5.1.3 Gestion des Participants

Ajouter/retirer, quitter canal, lister avec présence.

## 5.2 Module Messagerie

### 5.2.1 Messages

- Envoi texte (chemin principal : WS ; fallback : REST)
- Reply, Edit (auteur, délai configurable), Soft delete
- Pagination historique ; recherche full-text (GIN)
- Taille max : 4 096 caractères (configurable par plateforme)

### 5.2.2 Réactions Emoji

Ajouter/retirer, UNIQUE `(message_id, user_id, emoji)` en base, compteurs par emoji.

### 5.2.3 Accusés de Lecture

Curseur "lu jusqu'au message X", avancement auto, visibilité pour les participants. **Capacité configurable** (section 5.5).

## 5.3 Module Temps Réel (WebSocket)

### 5.3.1 Typing

Timeout 3s, Redis-only, capacité configurable.

### 5.3.2 Présence

En ligne / hors ligne / dernière connexion, Redis-only, capacité configurable.

### 5.3.3 Reconnexion

Auto-reconnexion, récupération messages manqués via REST, zéro perte (PostgreSQL avant WS).

## 5.4 Module Transfert Bot → Humain

- Chatbot appelle `POST /api/v1/chat/conversations/transfer` (S2S)
- Conversation créée `en_attente` ; opérateurs reçoivent `transfer:new`
- Opérateur prend en charge → `pris_en_charge`

— Opérateur clôture → `cloture` ; bot reprend
— File d'attente : message automatique si aucun opérateur
— Capacité configurable par plateforme

**Règles de prise en charge opérateur (v1.2) :**

— **Qui peut prendre en charge ?** Utilisateurs ayant la permission `chat:transfer:take` (vérifié via `GET /users/{id}/permissions/check`).
— **Concurrence :** verrou optimiste (`UPDATE ... WHERE status = 'en_attente'`). Deuxième opérateur reçoit `409 Conflict`.
— **Reprise :** un transfert `pris_en_charge` ne peut pas être repris. Si l'opérateur se déconnecte sans clôturer, job périodique remet en `en_attente` après délai configurable (défaut : 10 min).
— **Clôture :** uniquement par l'opérateur ayant pris en charge (`operator_id = user_id` du JWT).
— **Post-clôture :** conversation lisible en historique, plus de nouveaux messages.

## 5.5 Module Capacités par Plateforme

Le moteur Chat sait tout faire ; chaque plateforme active ce qu'elle veut. Config stockée en PostgreSQL, modifiable via API.

### 5.5.1 Capacités optionnelles

| Capacité | Description |
|---|---|
| `direct_enabled` | Conversations directes 1-to-1 |
| `channels_enabled` | Canaux (groupes) |
| `read_receipts_enabled` | Accusés lecture détaillés |
| `typing_enabled` | Indicateur saisie |
| `presence_enabled` | Présence en ligne |
| `reactions_enabled` | Réactions emoji |
| `transfer_enabled` | Transfert bot→humain |
| `message_edit_enabled` | Édition messages |
| `message_delete_enabled` | Suppression logique |
| `message_search_enabled` | Recherche full-text |
| `attachments_enabled` | Pièces jointes (futur) |

### 5.5.2 Capacités structurelles (toujours actives)

Auth obligatoire, isolation plateforme, persistance, sécurité, rate limiting, logs, contrôle d'accès.

### 5.5.3 Comportement désactivé

Endpoints → `403 Feature disabled` ; événements WS non émis ; coût technique évité.

### 5.5.4 Gouvernance des capacités (v1.2)

**Qui peut modifier `platform_capabilities` ?**

— **Lecture** : tout token JWT ou S2S valide de la plateforme.
— **Écriture** : réservée aux **administrateurs** :

1. JWT validé via `GET /auth/verify-token`.
2. Permission vérifiée via `GET /users/{user_id}/permissions/check?platform={p}&permis`

— La permission `chat:admin` doit être créée par chaque plateforme dans le Service Users.
— Alternative : token S2S avec `service_name = "admin"` (gestion automatisée).

# 6 Besoins Non-Fonctionnels

## 6.1 Performance

| Exigence | Cible |
|---|---|
| Latence WS | < 200ms |
| Connexions WS MVP | 1 000+ |
| Timeout typing | 3s |
| Chargement historique | < 300ms |
| Cache présence TTL | Configurable |
| Taille max message | 4 096 car. |

## 6.2 Sécurité

— Auth obligatoire via `GET /auth/verify-token`
— Isolation conversations par participant + `platform_id` du JWT
— Edit/Delete auteur uniquement ; soft delete obligatoire
— Rate limiting via Redis (section 6.4)

## 6.3 Scalabilité

Redis Adapter multi-instances, LB sticky-session, index PostgreSQL, Redis décharge.

## 6.4 Garde-fous

| Règle | Valeur |
|---|---|
| Taille max message | 4 096 car. |
| Rate limit / utilisateur | 30 msg/min |
| Rate limit / conversation | 100 msg/min |
| Anti-flood WS | 10 evt/s/client |
| Max membres canal | 500 |
| Archivage inactif | 12 mois |
| Backpressure WS | Livraison différée si file surchargée |
| Timeout reprise transfert | 10 min |

## 6.5 Disponibilité

≥ 99,9% ; persistance PG avant WS ; health check `GET /api/v1/chat/health` ; rolling updates avec drain WS.

# 7 Architecture Technique

### 7.1 Stack Technique

| Composant | Technologie | Rôle |
|---|---|---|
| Runtime | Node.js | Boucle événementielle temps réel |
| Framework API | Express.js | Endpoints REST |
| WebSocket | Socket.io | Temps réel, rooms |
| WS Adapter | Redis Adapter | Sync multi-instances |
| Base de données | PostgreSQL | Données durables |
| Cache | Redis | Présence, typing, sessions, non-lus |
| Broker | RabbitMQ | Async inter-services |
| Conteneurisation | Docker | Déploiement |

⚠ **Justification Node.js :** Exception documentée (comme Média/NestJS). Socket.io = référence temps réel ; boucle événementielle optimisée pour milliers de connexions ; Redis Adapter natif.

### 7.2 Intégrations Externes

| Service | Rôle | Phase |
|---|---|---|
| Auth v2.1 | JWT + S2S | MVP |
| Users v1.0 | Profils, permissions opérateur | MVP |
| Notification v1.2 | Alertes hors ligne | MVP |
| Chatbot | Création transferts (S2S) | MVP |
| Média v1.4 | Pièces jointes | Futur |

## 8 Modélisation

ℹ **Convention d'identité :** `user_id`, `sender_id`, `created_by`, `operator_id` = `users_auth.id` (= `sub` JWT). `platform_id` = UUID Auth. Présence et typing = Redis-only.

## 8.1 Table `conversations`

| Colonne | Type | Contr. | Description |
|---|---|---|---|
| `id` | UUID | PK | Identifiant |
| `type` | VARCHAR(2 | NOT NULL | direct/canal/transfert |
| `platform_id` | UUID | NOT NULL | FK logique → Auth |
| `name` | VARCHAR(2 | NULL | Nom (canaux) |
| `description` | TEXT | NULL | Description (canaux) |
| `is_public` | BOOLEAN | NULL | Visibilité (canaux) |
| `created_by` | UUID | NOT NULL | `users_auth.id` |
| `metadata` | JSONB | NULL | Métadonnées libres |
| `deleted_at` | TIMESTAMPTZ | NULL | Soft delete |
| `created_at` | TIMESTAMP | NOT NULL | Création |
| `updated_at` | TIMESTAMPTZ | NOT NULL | Dernière activité |

## 8.2 Table `conversation_participants`

PK composite `(conversation_id, user_id)`.

| Colonne | Type | Contr. | Description |
|---|---|---|---|
| `conversation_id` | UUID | PK, FK | Conversation |
| `user_id` | UUID | PK | Participant |
| `role` | VARCHAR(20) | NOT NULL | member/admin/owner |
| `joined_at` | TIMESTAMP | NOT NULL | Entrée |
| `left_at` | TIMESTAMPTZ | NULL | Sortie |

## 8.3 Table `messages`

Index : `(conversation_id, created_at DESC)`, GIN sur `content`.

| Colonne | Type | Contr. | Description |
|---|---|---|---|
| id | UUID | PK | Identifiant |
| conversation_id | UUID | FK, NN | Conversation |
| sender_id | UUID | NOT NULL | Auteur |
| content | TEXT | NULL | Contenu |
| parent_id | UUID | FK self | Reply |
| is_deleted | BOOLEAN | NOT NULL | Soft delete |
| edited_at | TIMESTAMPTZ | NULL | Modification |
| created_at | TIMESTAMP | NOT NULL | Envoi |

## 8.4 Table `message_reactions`

PK composite `(message_id, user_id, emoji)`.

| Colonne | Type | Contr. | Description |
|---|---|---|---|
| message_id | UUID | PK, FK | Message |
| user_id | UUID | PK | Utilisateur |
| emoji | VARCHAR(10) | PK | Code emoji |
| created_at | TIMESTAMP | NOT NULL | Date |

## 8.5 Table `read_cursors`

PK composite `(conversation_id, user_id)`.

| Colonne | Type | Contr. | Description |
|---|---|---|---|
| conversation_id | UUID | PK, FK | Conversation |
| user_id | UUID | PK | Participant |
| last_read_message_id | UUID | FK | Dernier lu |
| updated_at | TIMESTAMP | NOT NULL | MAJ |

### 8.6 Table transfers

| Colonne | Type | Contr. | Description |
|---|---|---|---|
| id | UUID | PK | Identifiant |
| conversation_id | UUID | FK, UQ | Conversation transfert |
| user_id | UUID | NOT NULL | Utilisateur transféré |
| operator_id | UUID | NULL | Opérateur |
| status | VARCHAR(20) | NOT NULL | en_attente/pris_en_charge/cloture |
| bot_history | JSONB | NOT NULL | Historique bot |
| context | JSONB | NULL | Contexte bot |
| taken_at | TIMESTAMP | NULL | Prise en charge |
| closed_at | TIMESTAMPTZ | NULL | Clôture |
| created_at | TIMESTAMP | NOT NULL | Création |

### 8.7 Table platform_capabilities

| Colonne | Type | Contr. | Description |
|---|---|---|---|
| platform_id | UUID | PK | FK logique Auth |
| direct_enabled | BOOLEAN | DEF true | Conv. directes |
| channels_enabled | BOOLEAN | DEF true | Canaux |
| read_receipts_enabled | BOOLEAN | DEF true | Accusés lecture |
| typing_enabled | BOOLEAN | DEF true | Typing |
| presence_enabled | BOOLEAN | DEF true | Présence |
| reactions_enabled | BOOLEAN | DEF true | Réactions |
| transfer_enabled | BOOLEAN | DEF false | Transfert |
| message_edit_enabled | BOOLEAN | DEF true | Édition |
| message_delete_enabled | BOOLEAN | DEF true | Suppression |
| message_search_enabled | BOOLEAN | DEF true | Recherche |
| attachments_enabled | BOOLEAN | DEF false | Pièces jointes |
| max_message_length | INTEGER | DEF 4096 | Taille max msg |
| rate_limit_per_user | INTEGER | DEF 30 | Msg/min/user |
| rate_limit_per_conv | INTEGER | DEF 100 | Msg/min/conv |
| max_channel_members | INTEGER | DEF 500 | Max membres |
| updated_at | TIMESTAMPTZ | NOT NULL | MAJ |

## 9 Contrats d'API

🔒 **Base URL :** `/api/v1/chat`. Token validé via Auth v2.1. `platform_id` et `user_id` extraits du JWT.

### 9.1 Matrice d'Autorisation (v1.2)

> ℹ **Pub.** = public. **JWT** = Bearer utilisateur. **S2S** = Service-to-Service. **Adm.** = JWT + permission `chat:admin` via Users, ou S2S `service_name="admin"`.

| Endpoint | M. | Pub | JWT | S2S | Adm |
|---|---|---|---|---|---|
| `/api/v1/chat/health` | GET | | | | |
| `/api/v1/chat/conversations` | POST | | | | |
| `/api/v1/chat/conversations` | GET | | | | |
| `/api/v1/chat/conversations/{id}` | GET | | | | |
| `/api/v1/chat/conversations/{id}` | PUT | | | | |
| `/api/v1/chat/conversations/{id}` | DEL | | | | |
| `.../{id}/participants` | POST | | | | |
| `.../{id}/participants/{uid}` | DEL | | | | |
| `.../{id}/participants` | GET | | | | |
| `.../{id}/leave` | POST | | | | |
| `.../{id}/messages` | POST | | | | |
| `.../{id}/messages` | GET | | | | |
| `.../{id}/messages/{msgId}` | PUT | | | | |
| `.../{id}/messages/{msgId}` | DEL | | | | |
| `.../{id}/messages/search` | GET | | | | |
| `.../{msgId}/reactions` | POST | | | | |
| `.../{msgId}/reactions/{emoji}` | DEL | | | | |
| `.../{msgId}/reactions` | GET | | | | |
| `.../{id}/read` | POST | | | | |
| `.../{id}/read-status` | GET | | | | |
| `/api/v1/chat/conversations/transfer` | POST | | | | |
| `.../transfers/pending` | GET | | | | |
| `.../transfers/{id}/take` | POST | | | | |
| `.../transfers/{id}/close` | POST | | | | |
| `.../platforms/{p}/channels` | GET | | | | |
| `/api/v1/chat/channels/{id}/join` | POST | | | | |
| `.../users/{uid}/presence` | GET | | | | |
| `.../conversations/stats` | GET | | | | |
| `.../transfers/stats` | GET | | | | |
| `.../capabilities/{p}` | GET | | | | |
| `.../capabilities/{p}` | PUT | | | | |

*Note : les chemins abrégés* `.../` *sont relatifs à* `/api/v1/chat/conversations` *sauf indication contraire.*

## 9.2 Endpoints REST

Les endpoints sont identiques à la v1.1 avec les corrections de chemin canonique. Tous utilisent le préfixe `/api/v1/chat`. Les endpoints liés aux capacités désactivées retournent `403 Feature disabled`. Le transfert `POST .../transfer` est S2S uniquement. Les endpoints `.../transfers/{id}/take` vérifient la permission `chat:transfer:take` via Users et utilisent un verrou optimiste (409 Conflict). Détails complets des body/response : voir v1.1 section 9.

## 9.3 Événements WebSocket

Connexion : token Bearer → validation Auth → extraction `user_id`/`platform_id` → chargement capabilities → inscription rooms. Événements des capacités désactivées non émis. WS = chemin principal envoi ; REST = fallback. Persistance PG avant émission.

### 9.3.1 Serveur → Client

| Événement | Payload & Condition |
|---|---|
| `message:new` | `{message_id, conversation_id, sender_id, content, parent_id, created_at}` |
| `message:updated` | `{message_id, content, edited_at}` |
| `message:deleted` | `{message_id}` |
| `typing:start` | si `typing_enabled` |
| `typing:stop` | si `typing_enabled` |
| `read:update` | si `read_receipts_enabled` |
| `reaction:new` | si `reactions_enabled` |
| `reaction:removed` | si `reactions_enabled` |
| `participant:online` | si `presence_enabled` |
| `participant:offline` | si `presence_enabled` |
| `transfer:new` | si `transfer_enabled` |
| `conversation:updated` | toujours |

### 9.3.2 Client → Serveur

`message:send`, `typing:start`, `typing:stop`, `message:read`, `reaction:add`, `reaction:remove`. Payloads identiques v1.1.

# 10 Contrats Inter-Services

## 10.1 Chat → Auth (JWT)

Endpoint `GET /auth/verify-token`. Cache Redis TTL 30s. Échec → 401.

## 10.2 Chat → Auth (S2S)

Endpoint `POST /auth/s2s/introspect`. Usage : Chatbot transfert, admin capabilities.

## 10.3 Chat → Users

| Param. | Valeur |
|---|---|
| Endpoint profil | `GET /users/{users_auth.id}` |
| Usage profil | Nom, photo, `users_profiles.id` |
| Endpoint permissions | `GET /users/{id}/permissions/check` |
| Usage permissions | `chat:transfer:take`, `chat:admin` |
| Cache | TTL 5 min (profil), 30s (permissions) |
| Si Users down | Profil : “Utilisateur inconnu”. Permissions : refus (fail-closed). |

### 10.4 Chat → Notification

| Param. | Valeur |
|---|---|
| Endpoint | `POST /api/v1/notifications/send` |
| Auth | Token S2S |
| Catégorie | `transactional`, priorité `normal` |
| Échec | Log ; ne bloque jamais l'envoi du message |

**⚠ Mapping identité Chat → Notification (v1.2) :**

Chat travaille avec `users_auth.id`. Notification attend `users_profiles.id`.

**Séquence :**

1. Chat détecte destinataire hors ligne.
2. Chat appelle `GET /users/{users_auth.id}` → résout `users_profiles.id` (déjà en cache Redis TTL 5 min).
3. Chat appelle Notification avec `user_id = users_profiles.id`.

Chat n'envoie **jamais** un `users_auth.id` brut à Notification.

#### 10.4.1 Templates Notification pré-requis

| Template | Canal | Déclencheur |
|---|---|---|
| `chat_message_offline` | push | Message hors ligne |
| `chat_transfer_new` | push | Nouveau transfert |
| `chat_transfer_assigned` | push | Transfert pris en charge |
| `chat_transfer_closed` | push | Transfert clôturé |

### 10.5 Chatbot → Chat

`POST /api/v1/chat/conversations/transfer`. Auth S2S. Données : `user_id` (`users_auth.id`), `platform_id`, `bot_history`, `context`.

### 10.6 Chat → Média (futur)

`GET /api/v1/media/{id}`. Quand `attachments_enabled = true`. Messages avec `media_ids : UUID[]`.

## 11 Infrastructure

### 11.1 Composants

PostgreSQL, Redis, Socket.io + Redis Adapter, RabbitMQ, Docker, LB sticky-session, index GIN. Tous MVP.

### 11.2 Variables d'Environnement

| Variable | Description |
|---|---|
| DATABASE_URL | PostgreSQL |
| REDIS_URL | Redis |
| AUTH_SERVICE_URL | Auth v2.1 |
| USERS_SERVICE_URL | Users v1.0 |
| NOTIFICATION_SERVICE_URL | Notification v1.2 |
| BROKER_URL | RabbitMQ |
| SOCKET_IO_CORS_ORIGIN | Origines WS |
| TYPING_TIMEOUT_MS | Défaut 3000 |
| MESSAGE_EDIT_DELAY_MIN | Délai edit (min) |
| TRANSFER_TIMEOUT_MIN | Défaut 10 |
| SECRET_KEY | Clé Node.js |
| USERS_CACHE_TTL_SEC | Défaut 300 |
| AUTH_CACHE_TTL_SEC | Défaut 30 |

### 11.3 Docker Compose

Listing 1 – docker-compose.yml

```yaml
services:
  chat-service:
    build: ./chat-service
    ports: ["3005:3005"]
    environment:
      - DATABASE_URL=postgresql://...
      - REDIS_URL=redis://redis:6379
      - AUTH_SERVICE_URL=http://auth-service:3001
      - USERS_SERVICE_URL=http://user-service:3002
      - NOTIFICATION_SERVICE_URL=http://notif-service:3004
      - BROKER_URL=amqp://rabbitmq:5672
    depends_on: [postgres, redis, rabbitmq]
  postgres:
    image: postgres:16-alpine
    volumes: [chat_pg_data:/var/lib/postgresql/data]
  redis:
    image: redis:7-alpine
  rabbitmq:
    image: rabbitmq:3-management
volumes:
  chat_pg_data:
```

## 12 Recommandations & Évolutions

### 12.1 Scaling WebSocket

Multi-instances Socket.io + LB sticky-session + Redis Adapter.

### 12.2 Pièces Jointes

Champ `media_ids` via Service Média. Chat ne stocke jamais les fichiers.

## 12.3 E2E

Signal Protocol pour conversations directes sensibles.

## 12.4 Archivage

Conversations inactives 12 mois+ vers `messages_archive`. Job périodique. Lecture seule.

## 12.5 Suppression

Soft delete message (auteur), soft delete conversation (admin), archivage (inactif).

# A Annexe A — Diagramme de Classes (PlantUML)

Listing 2 – PlantUML — Classes Chat Service v1.2

```
@startuml Chat Service - Class Diagram v1.2
skinparam classAttributeIconSize 0
skinparam linetype ortho

entity "conversations" as conv {
  * id : UUID <<PK>>
  --
  * type : VARCHAR(20)
  * platform_id : UUID <<FK logique -> Auth>>
  name : VARCHAR(255)
  description : TEXT
  is_public : BOOLEAN
  * created_by : UUID
  metadata : JSONB
  deleted_at : TIMESTAMPTZ
  * created_at : TIMESTAMPTZ
  * updated_at : TIMESTAMPTZ
}

entity "conversation_participants" as part {
  * conversation_id : UUID <<PK, FK>>
  * user_id : UUID <<PK>>
  --
  * role : VARCHAR(20)
  * joined_at : TIMESTAMPTZ
  left_at : TIMESTAMPTZ
}

entity "messages" as msg {
  * id : UUID <<PK>>
  --
  * conversation_id : UUID <<FK>>
  * sender_id : UUID
  content : TEXT
  parent_id : UUID <<FK self>>
  * is_deleted : BOOLEAN
  edited_at : TIMESTAMPTZ
  * created_at : TIMESTAMPTZ
}

entity "message_reactions" as react {
  * message_id : UUID <<PK, FK>>
  * user_id : UUID <<PK>>
  * emoji : VARCHAR(10) <<PK>>
  --
  * created_at : TIMESTAMPTZ
}

entity "read_cursors" as cursor {
  * conversation_id : UUID <<PK, FK>>
  * user_id : UUID <<PK>>
  --
  last_read_message_id : UUID <<FK>>
  * updated_at : TIMESTAMPTZ
}

entity "transfers" as xfer {
  * id : UUID <<PK>>
  --
  * conversation_id : UUID <<FK, UNIQUE>>
  * user_id : UUID
  operator_id : UUID
  * status : VARCHAR(20)
  * bot_history : JSONB
  context : JSONB
  taken_at : TIMESTAMPTZ
  closed_at : TIMESTAMPTZ
  * created_at : TIMESTAMPTZ
}
```

```

entity "platform_capabilities" as cap {
  * platform_id : UUID <<PK>>
  --
  direct_enabled : BOOLEAN = true
  channels_enabled : BOOLEAN = true
  read_receipts_enabled : BOOLEAN = true
  typing_enabled : BOOLEAN = true
  presence_enabled : BOOLEAN = true
  reactions_enabled : BOOLEAN = true
  transfer_enabled : BOOLEAN = false
  message_edit_enabled : BOOLEAN = true
  message_delete_enabled : BOOLEAN = true
  message_search_enabled : BOOLEAN = true
  attachments_enabled : BOOLEAN = false
  max_message_length : INT = 4096
  rate_limit_per_user : INT = 30
  rate_limit_per_conv : INT = 100
  max_channel_members : INT = 500
  * updated_at : TIMESTAMPTZ
}

note as N1
  <b>Redis-only</b>
  Presence : user_id, status, last_seen_at
  Typing : conversation_id, user_id
end note

conv "1" -- "*" part
conv "1" -- "*" msg
conv "1" -- "0..1" xfer
conv "1" -- "*" cursor
msg "1" -- "*" react
msg "0..1" -- "*" msg : parent_id
@enduml
```

# B Annexe B — MLD (PlantUML)

Listing 3 – PlantUML — MLD Chat Service v1.2

```
@startuml Chat Service - MLD v1.2
skinparam classAttributeIconSize 0
skinparam linetype ortho

entity "conversations" as conv {
  * id : UUID <<PK>>
  --
  * type : VARCHAR(20)
  * platform_id : UUID
  name : VARCHAR(255)
  description : TEXT
  is_public : BOOLEAN
  * created_by : UUID
  metadata : JSONB
  deleted_at : TIMESTAMPTZ
  * created_at : TIMESTAMPTZ
  * updated_at : TIMESTAMPTZ
}
entity "conversation_participants" as part {
  * conversation_id : UUID <<PK, FK>>
  * user_id : UUID <<PK>>
  --
  * role : VARCHAR(20)
  * joined_at : TIMESTAMPTZ
  left_at : TIMESTAMPTZ
}
entity "messages" as msg {
  * id : UUID <<PK>>
  --
  * conversation_id : UUID <<FK>>
```

```
  * sender_id : UUID
  content : TEXT
  parent_id : UUID <<FK self>>
  * is_deleted : BOOLEAN = false
  edited_at : TIMESTAMPTZ
  * created_at : TIMESTAMPTZ
  ..
  <<INDEX>> (conversation_id, created_at DESC)
  <<GIN>> content
}
entity "message_reactions" as react {
  * message_id : UUID <<PK, FK>>
  * user_id : UUID <<PK>>
  * emoji : VARCHAR(10) <<PK>>
  --
  * created_at : TIMESTAMPTZ
}
entity "read_cursors" as cursor {
  * conversation_id : UUID <<PK, FK>>
  * user_id : UUID <<PK>>
  --
  last_read_message_id : UUID <<FK>>
  * updated_at : TIMESTAMPTZ
}
entity "transfers" as xfer {
  * id : UUID <<PK>>
  --
  * conversation_id : UUID <<FK, UNIQUE>>
  * user_id : UUID
  operator_id : UUID
  * status : VARCHAR(20)
  * bot_history : JSONB
  context : JSONB
  taken_at : TIMESTAMPTZ
  closed_at : TIMESTAMPTZ
  * created_at : TIMESTAMPTZ
}
entity "platform_capabilities" as cap {
  * platform_id : UUID <<PK>>
  --
  (all feature flags + limits)
  * updated_at : TIMESTAMPTZ
}
conv ||--o{ part
conv ||--o{ msg
conv ||--o| xfer
conv ||--o{ cursor
msg ||--o{ react
msg |o--o{ msg : parent_id
@enduml
```

# Annexe C — Glossaire

| Terme | Définition |
|---|---|
| WebSocket | Communication bidirectionnelle persistante sur TCP |
| Socket.io | Librairie WebSocket avec fallback et rooms |
| Room | Canal virtuel Socket.io par conversation |
| Redis Adapter | Sync événements entre instances Socket.io |
| Pub/Sub | Diffusion événements Redis inter-processus |
| Sticky Session | Affinité LB client→instance |
| S2S | Service-to-Service auth |
| Feature Flag | Capacité activable/désactivable par plateforme |
| Typing | "X est en train d'écrire" |
| Curseur de lecture | Dernier message lu par participant |
| Transfert | Conversation bot→humain |
| Soft delete | Suppression logique |
| E2E | Chiffrement bout en bout |
| TTL | Durée de vie cache Redis |
| Backpressure | Contrôle flux si surcharge |
| `users_auth.id` | ID technique Auth (= `sub` JWT) |
| `users_profiles.id` | ID profil Users (utilisé par Notification) |
| Verrou optimiste | UPDATE conditionnel (pas de lock) |

**AG Technologies** — Document confidentiel à usage interne exclusivement.
*Chat Service CDC v1.2 — 9 avril 2026*